# CAHIER DES CHARGES TECHNIQUE

Projet : Application Web de gestion des JO 2028

Langages : PHP – MySQL – HTML – CSS – JavaScript

Méthodologie : Agile Kanban (Trello)

Imad Chattou

BTS SIO – 2024/2025

# Sommaire


1. Contexte du projet
    1.1. Présentation du projet
    1.2. Date de rendu du projet
2. Besoins fonctionnels
3. Ressources nécessaires à la réalisation du projet
    3.1. Ressources matérielles
    3.2. Ressources logicielles
4. Gestion du projet
5.Plateforme collaborative de développement
6. Conception du projet
    6.1. Le front-end
        6.1.1. Wireframes
        6.1.2. Maquettes
        6.1.3. Arborescences
    6.2. Le back-end
        6.2.1. Diagramme de cas d'utilisation
        6.2.2. Diagramme d'activités
        6.2.3. Modèles Conceptuel de Données (MCD)
        6.2.4. Modèle Logique de Données (MLD)
        6.2.5. Modèle Physique de Données (MPD)
7. Technologies utilisées
    7.1. Langages de développement Web
    7.2. Base de données
8. Sécurité
    8.1. Login et protection des pages administrateurs
    8.2. Hachages des mots de passe avec Bcrypt
    8.3. Protection contre les attaques XSS (Cross-Site Scripting)
    8.4. Protection contre les injections SQL

# 1. Contexte du projet

## 1.1. Présentation du projet

Votre agence web a été sélectionnée par le comité d'organisation des jeux olympiques de Los Angeles 2028 pour développer une application web permettant aux organisateurs, aux médias et aux spectateurs de consulter des informations sur les sports, les calendriers des épreuves et les résultats des JO 2028.

Votre équipe et vous-même avez pour mission de proposer une solution qui répondra à la demande du client.

## 1.2. Date de rendu du projet

Le projet doit être rendu au plus tard le 15/01/2025.

# 2. Besoins fonctionnels

Le site web devra avoir une partie accessible au public et une partie privée permettant de gérer les données.

Les données seront stockées dans une base de données relationnelle pour faciliter la gestion et la mise à jour des informations. Ces données peuvent être gérées directement via le site web à travers un espace administrateur.

# 3. Ressources nécessaires à la réalisation du projet

## 3.1. Ressources matérielles

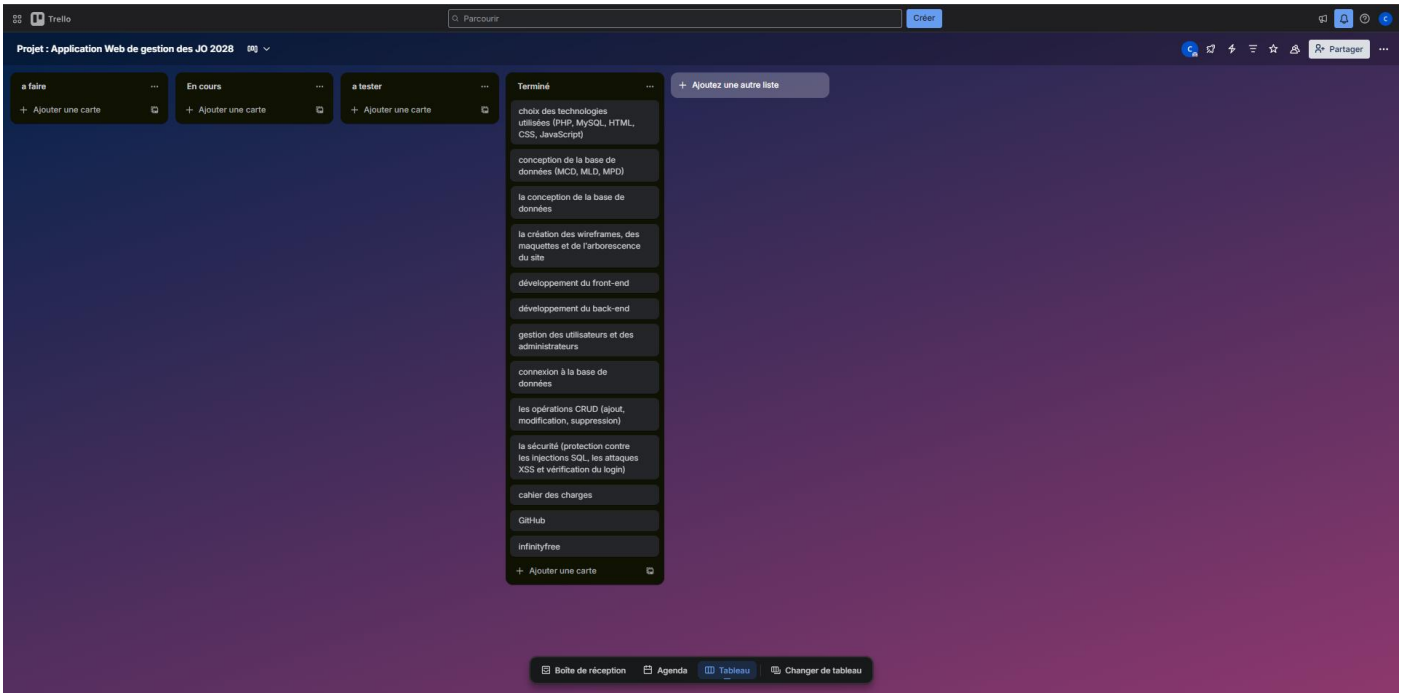
- PC
- Connexion internet
- Serveur local
- Périphérique de travail

## 3.2. Ressources logicielles

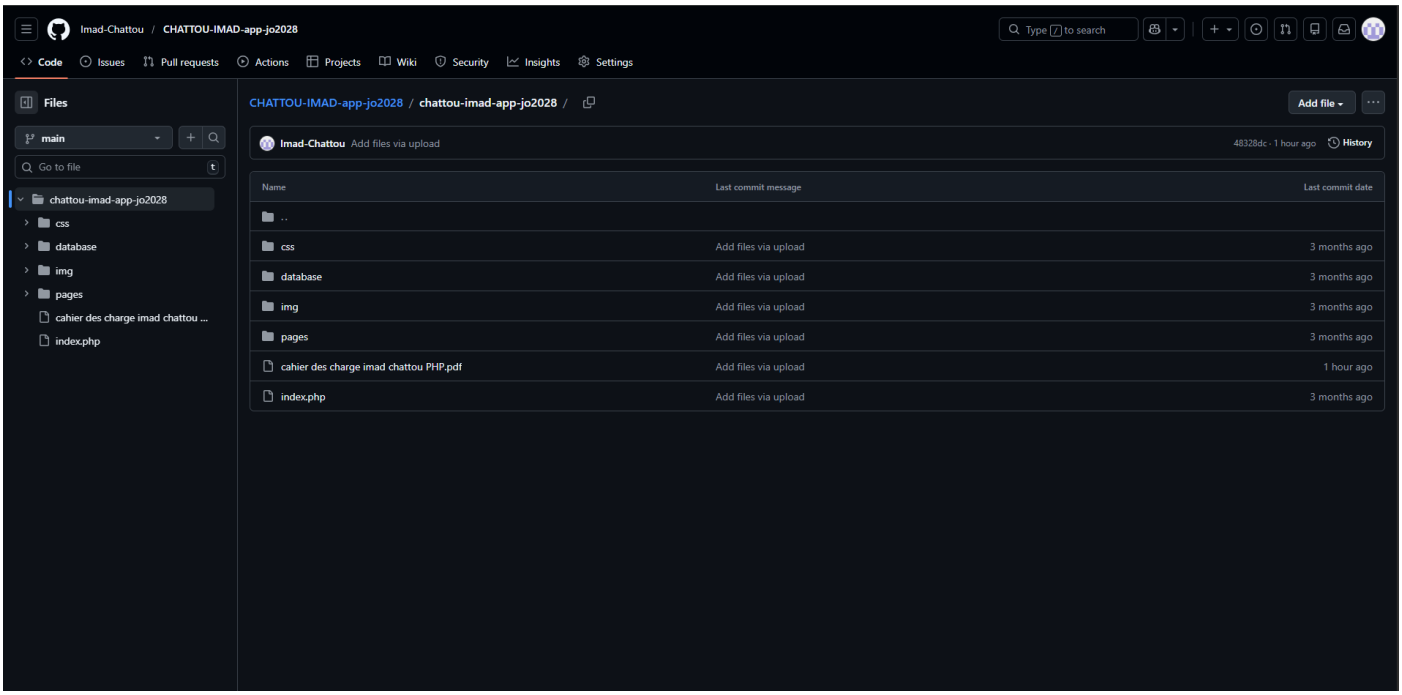
- Visual studio code
- Php
- Mysql
- Mamp
- Mocodo
- Java-script
- Html-CSS

# 4. Gestion du projet

Pour réaliser le projet, nous utiliserons la méthode Agile Kanban. Nous utiliserons également l'outil de gestion de projet en ligne Trello.




# 5. Plateforme collaborative de développement

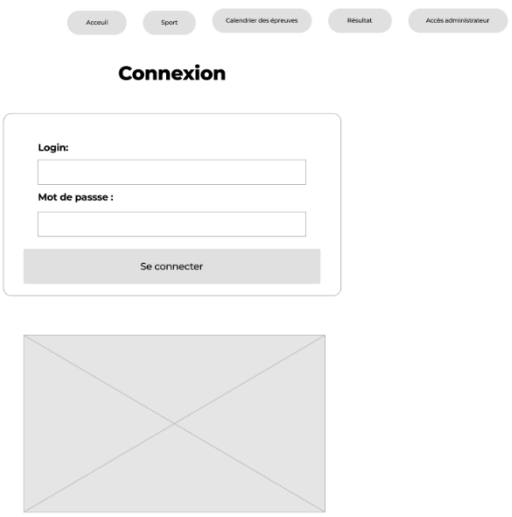


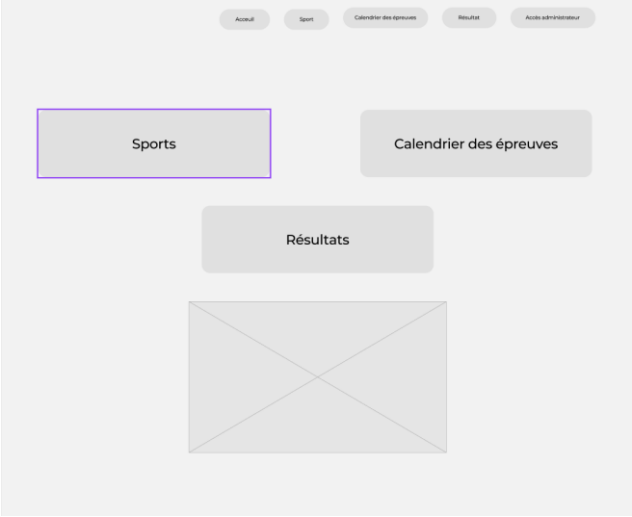
# 6. Conception du projet

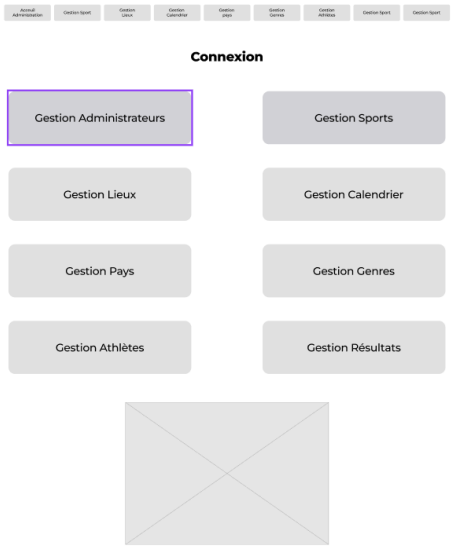
## 6.1. Le front-end

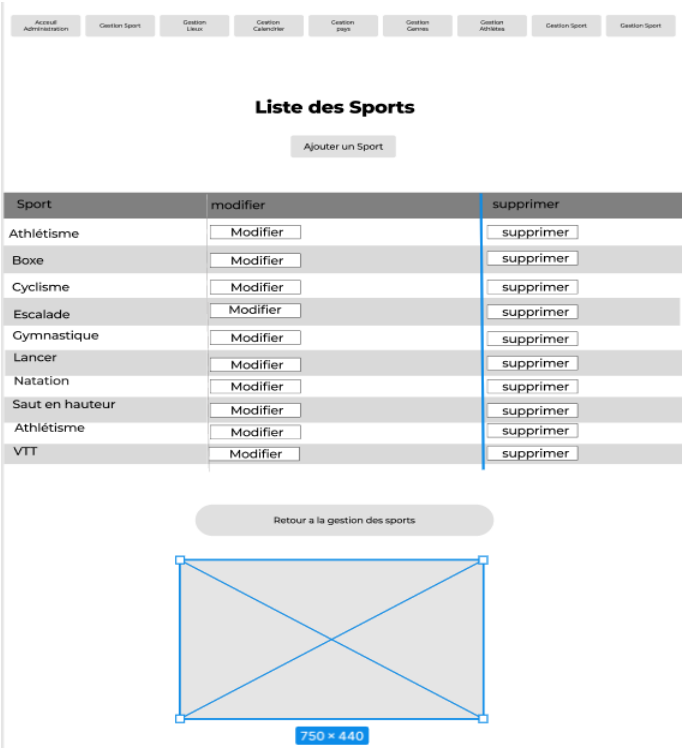
### 6.1.1. Wireframes

Version Ordinateur :

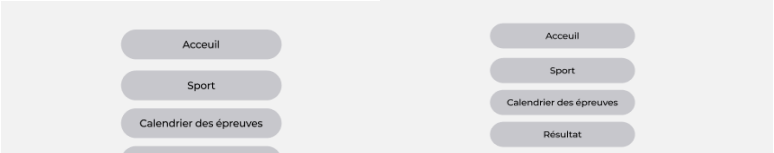


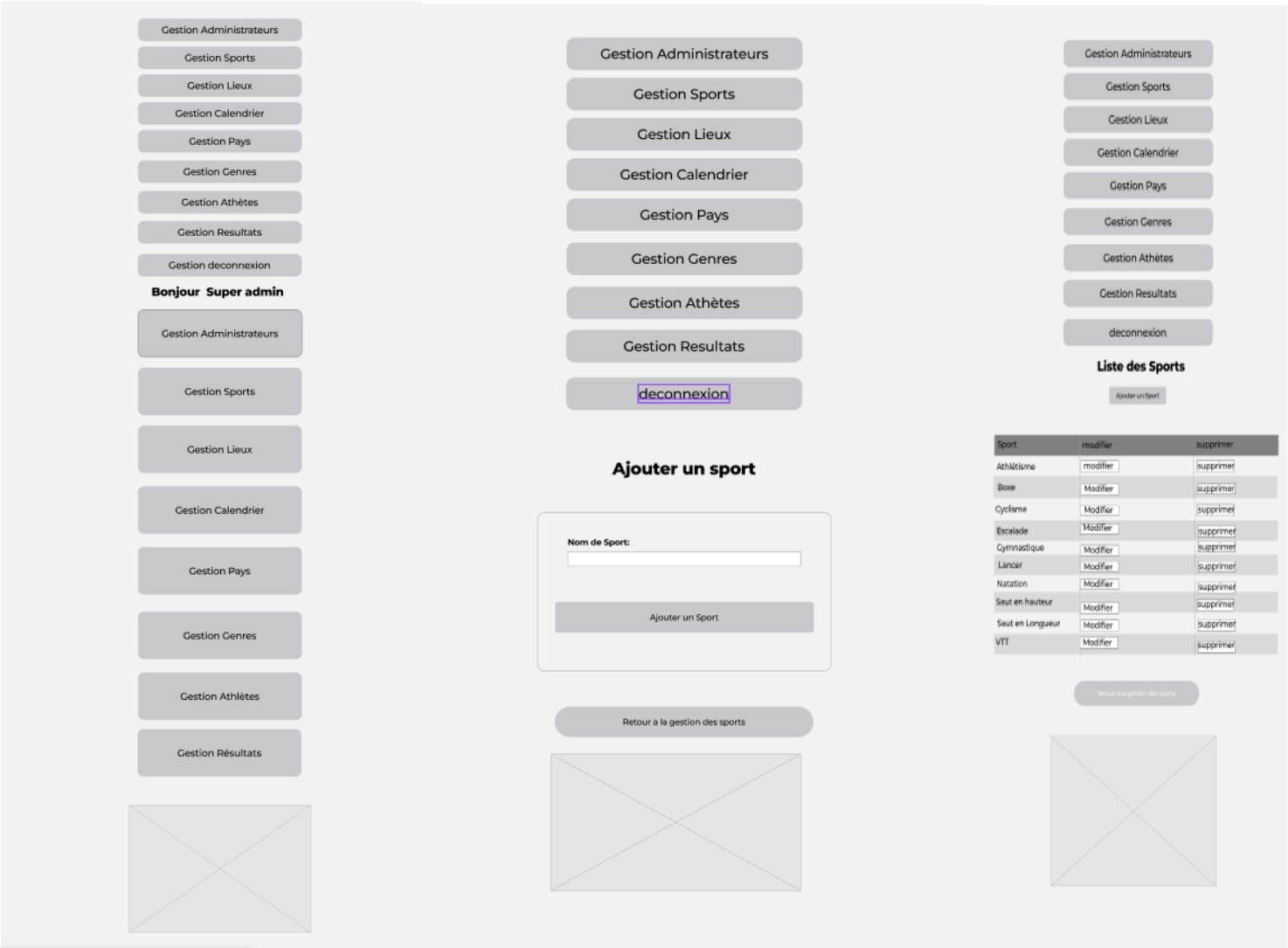







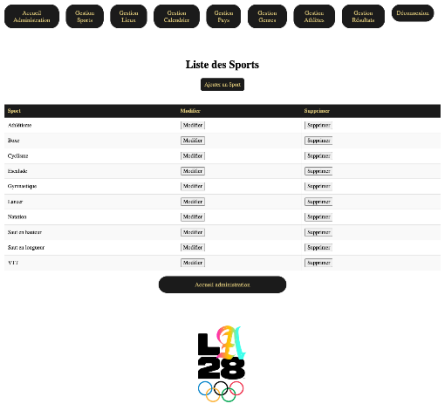

Version Téléphone :

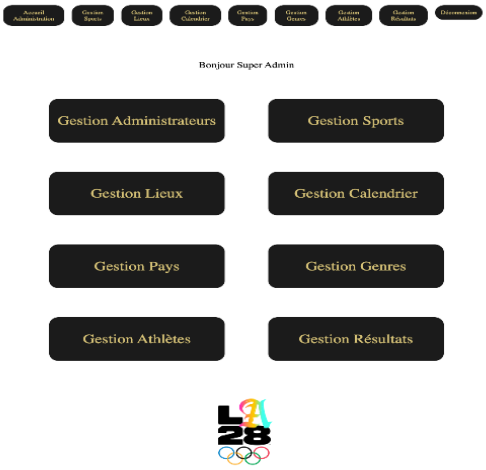


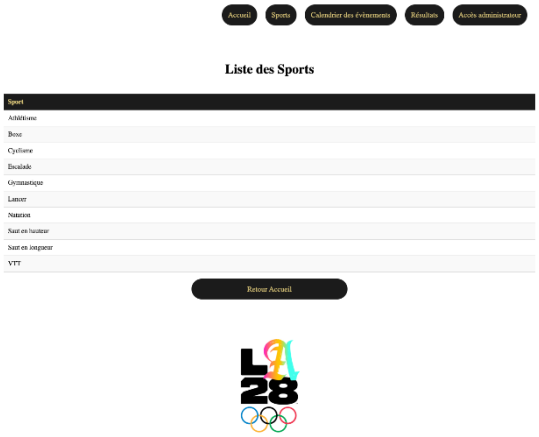



### 6.1.2. Maquettes

Version Ordinateur :





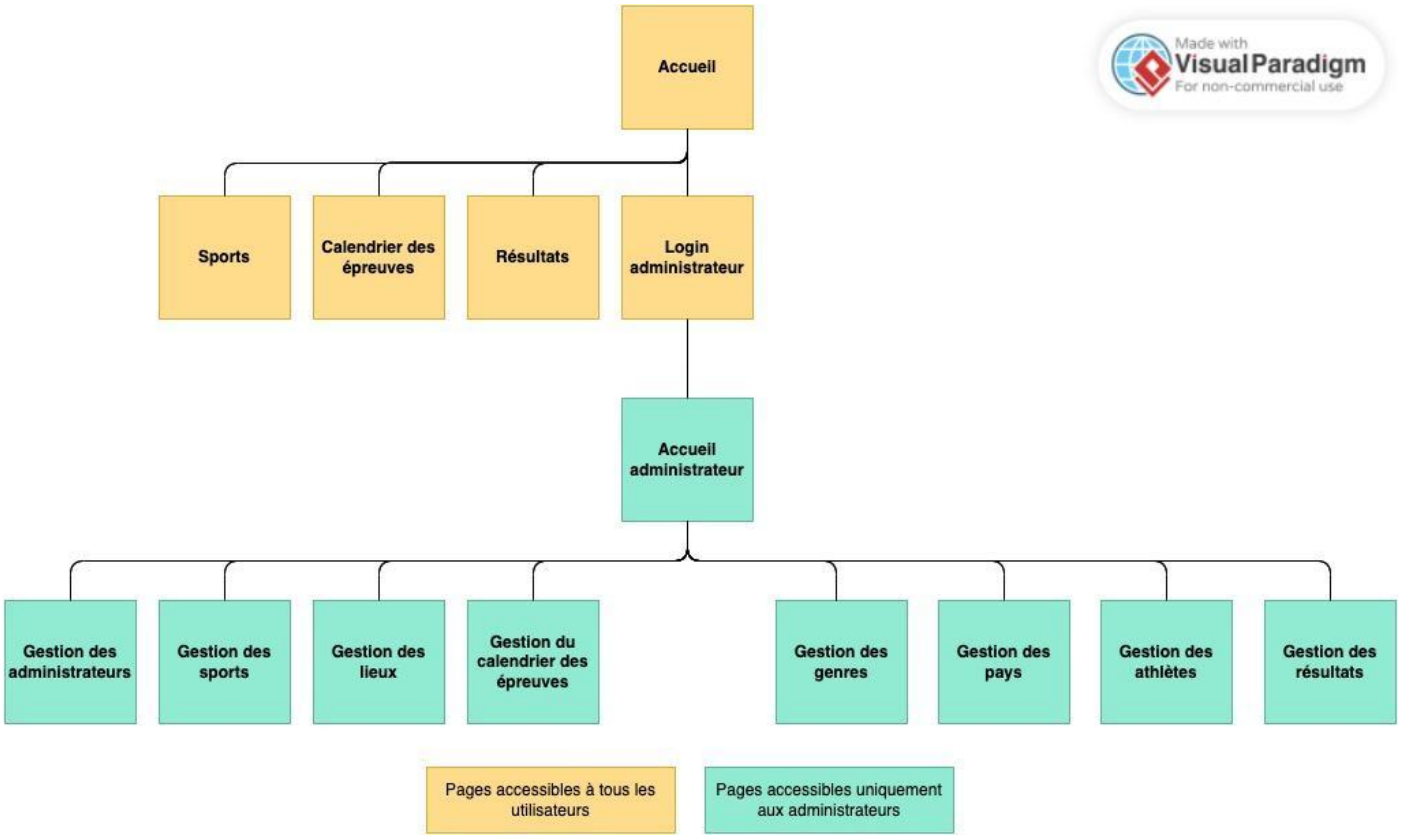



<u>Version Téléphone :</u>

Im

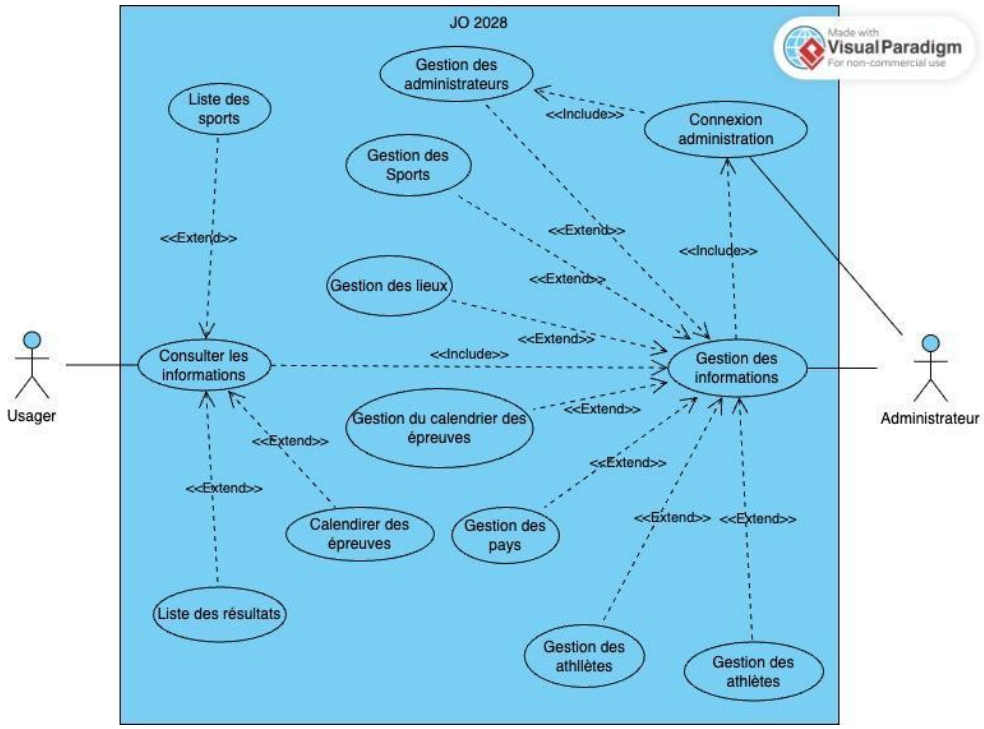
s

### 6.1.3. Arborescences

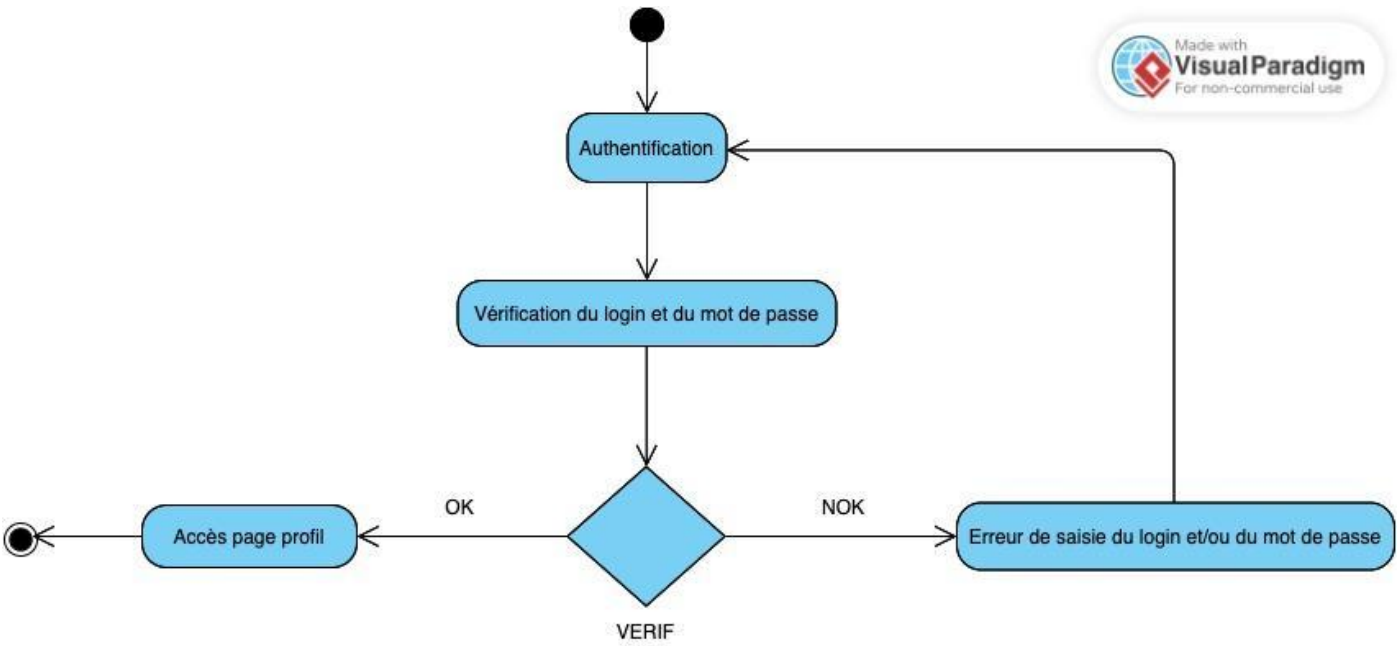



## 6.2. Le back-end

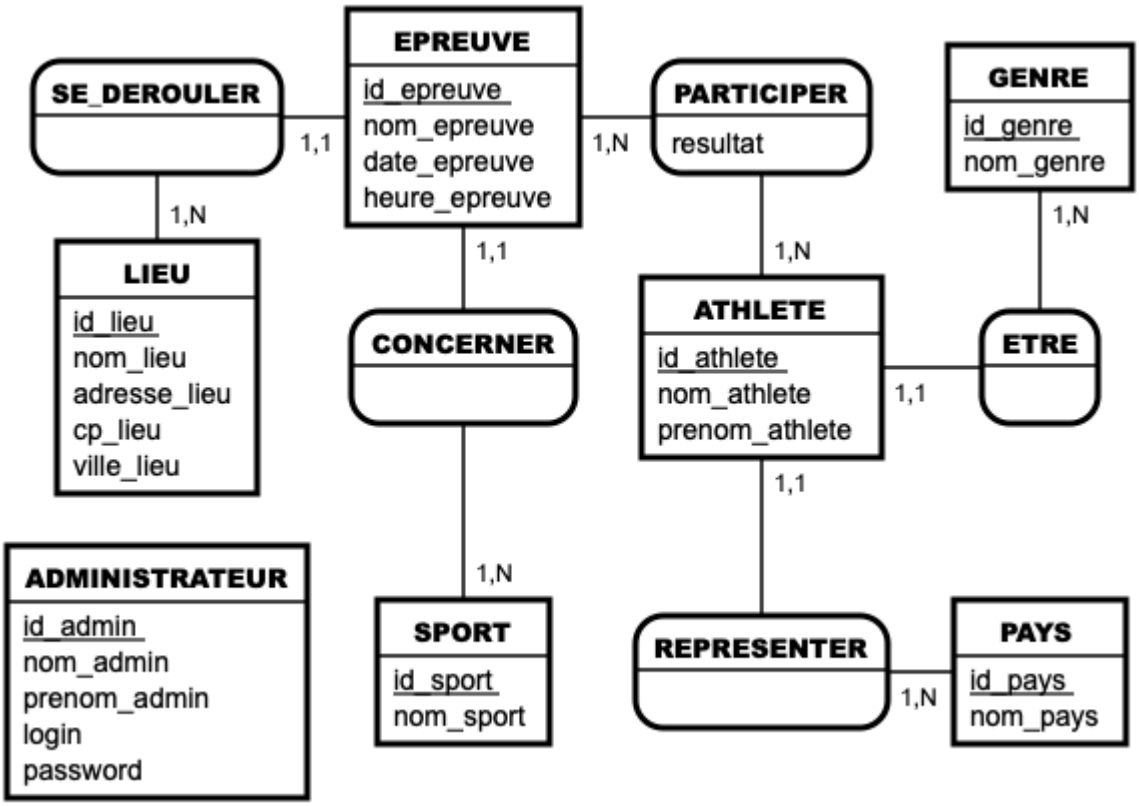
### 6.2.1. Diagramme de cas d'utilisation



### 6.2.2. Diagramme d'activités




### 6.2.3. Modèles Conceptuel de Données (MCD)



### 6.2.4. Modèle Logique de Données (MLD)

- ATHLETE (id_athlete, nom_athlete, prenom_athlete, id_pays, id_genre)
  Clé primaire : id_athlete
  Clés étrangères : id_pays en référence à id_pays de PAYS id_genre en référence à i d_genre de GENRE

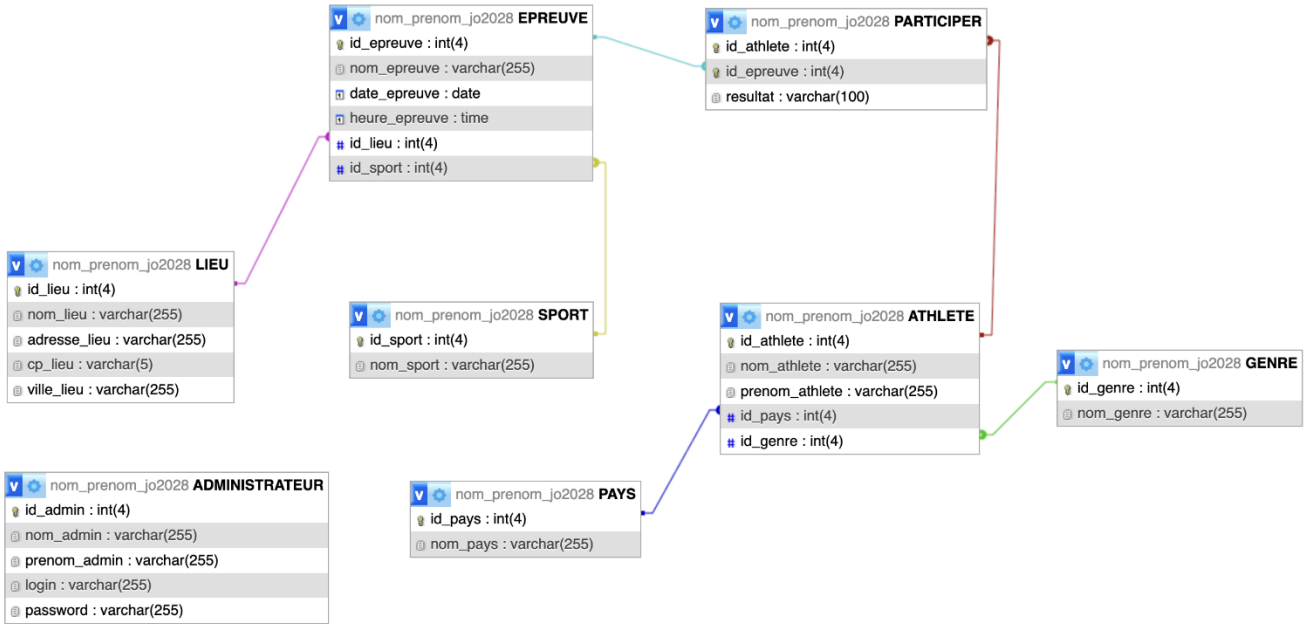
- EPREUVE (id_epreuve, nom_epreuve, date_epreuve, heure_epreuve, id_lieu, id_sport)
  Clé primaire : id_epreuve
  Clés étrangères : id_lieu en référence à id_lieu de LIEU id_sport en référence à id_sport de SPORT

- GENRE (id_genre, nom_genre)
  Clé primaire : id_genre

- LIEU (id_lieu, nom_lieu, adresse_lieu, cp_lieu, ville_lieu)
  Clé primaire : id_lieu

- PARTICIPER (id_athlete, id_epreuve, resultat)
  Clé primaire : id_athlete, id_epreuve
  Clés étrangères : id_athlete en référence à id_athlete de ATHLETE id_epreuve en référence à id_epreuve de EPREUVE
- PAYS (id_pays, nom_pays)
  Clé primaire : id_pays, id_epreuve
- SPORT (id_sport, nom_sport)
  Clé primaire : id_sport
- UTILISATEUR (id_utilisateur, nom_utilisateur, prenom_utilisateur, login, password)
  Clé primaire : id_utilisateur
- ADMINISTRATEUR (id_admin, nom_admin, prenom_admin, login, password)
  Clé primaire : id_admin

### 6.2.5. Modèle Physique d Données (MPD)




# 7. Technologies utilisées

## 7.1. Langages de développement Web

## 7.2. Base de données

# 8. Sécurité

## 8.1. Login et protection des pages administrateurs

### Définition de la solution :

La solution consiste à sécuriser l'accès aux pages administratives en utilisant la gestion des sessions PHP et la protection CSRF.
Seuls les utilisateurs authentifiés peuvent accéder aux pages sensibles et effectuer des actions telles que l'ajout, la modification ou la suppression de données. Les données sont filtrées et les requêtes à la base de données utilisent des requêtes préparées pour éviter les injections SQL.

### Explication du fonctionnement

1. **Authentification de l'utilisateur**

- L'utilisateur saisit son login et son mot de passe dans un formulaire sécurisé.
- Un **token CSRF** est généré pour protéger le formulaire contre les attaques de type Cross-Site Request Forgery.

2. **Vérification des identifiants**

- Les informations sont comparées à celles stockées dans la base de données via PDO.
- Si corrects, les informations de l'utilisateur sont stockées dans $_SESSION pour maintenir la session active.

3. **Protection des pages administratives**

- Chaque page commence par session_start() et vérifie que $_SESSION['login'] existe.
- Si la session n'existe pas, l'utilisateur est redirigé vers la page de connexion.

4. **Sécurisation des actions sensibles**

- Les actions POST (ajout, modification, suppression) vérifient le **token CSRF**.
- Les données sont filtrées avec filter_input() ou htmlspecialchars() avant d'être envoyées à la base de données.

## Extraits de code

**Vérification de connexion sur une page admin :**

```
session_start();
if (!isset($_SESSION['login'])) {
    header('Location: ../../../index.php');
    exit();
}
```

**Génération et vérification du token CSRF :**

```
// Générer un token CSRF
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}

// Vérifier le token CSRF lors d'une requête POST :
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    if (!isset($_POST['csrf_token']) ||
        !hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'])) {
        die('Token CSRF invalide.');
    }
}
```

**Exemple d'action sécurisée (suppression d'un sport) :**

```
$sql = "DELETE FROM SPORT WHERE id_sport = :param_id_sport";
$statement = $connexion->prepare($sql);
$statement->bindParam(':param_id_sport', $id_sport, PDO::PARAM_INT);
$statement->execute();
```

## 8.2. Hachages des mots de passe avec Bcrypt

### Définition de la solution :

**Bcrypt** est un algorithme de hachage spécialement conçu pour sécuriser les mots de passe.
Il sert à transformer un mot de passe en une valeur illisible et impossible à retrouver directement.
**Pourquoi on utilise Bcrypt ?**

- Impossible de retrouver le mot de passe original
- Algorithme lent = protection contre les attaques par force brute
- Utilise un “salt” automatique pour empêcher les attaques par tables arc-en-ciel
- Standard moderne recommandé en cybersécurité
- Prend en charge le “cost”, donc on peut augmenter la sécurité quand les machines deviennent plus puissantes

Bcrypt protège donc les utilisateurs même en cas de fuite de la base de données.

**Comment l’utiliser sur PHP ?**
PHP simplifie l’utilisation de Bcrypt grâce aux fonctions :

- password_hash() → pour hacher un mot de passe
- password_verify() → pour vérifier un mot de passe

Ces deux fonctions utilisent par défaut **PASSWORD_BCRYPT**.

### Extraits de code

**Hachage du mot de passe lors de l’inscription :**

```
$password = $_POST['password'];

// Hachage avec bcrypt
$hash = password_hash($password, PASSWORD_BCRYPT);

// On stocke $hash en base, jamais le mot de passe
```

**Vérification lors de la connexion :**

```
// $hash provient de la base de données
if (password_verify($_POST['password'], $hash)) {
    // Mot de passe correct → connexion autorisée
    $_SESSION['login'] = $user['login'];
} else {
    // Mot de passe incorrect
    $_SESSION['error'] = "Identifiants incorrects.";
}
```

# 8.3. Protection contre les attaques XSS (Cross-Site Scripting)

## Définition de la solution :

Les attaques **XSS (Cross-Site Scripting)** se produisent lorsqu'un utilisateur malveillant injecte du code HTML ou JavaScript dans une application web. L'objectif est de voler des informations sensibles, modifier l'affichage d'une page ou rediriger les utilisateurs vers un site dangereux.

**Comment nous protégeons l'application**

Dans notre application (gestion des sports et accès administrateurs) :

1. **Affichage sécurisé des données**
   - Tous les contenus affichés qui proviennent de la base de données ou de l'utilisateur (comme les noms de sports ou les messages d'erreur) sont **échappés** avant d'être affichés.
   - Cela empêche l'exécution de tout code malveillant dans le navigateur de l'utilisateur.
2. **Filtrage des données entrantes**
   - Les informations saisies dans les formulaires (ajout ou modification de sport, login) sont **nettoyées** avant d'être stockées ou traitées.
   - Cette étape empêche l'injection de balises HTML ou de scripts dangereux dans la base de données.
3. **Validation constante des entrées**
   - Nous considérons toutes les données venant de l'utilisateur comme potentiellement dangereuses et les validons systématiquement avant utilisation.
   - Les messages d'erreur affichés côté client sont également sécurisés pour éviter toute manipulation malveillante.

## Extraits de code

**Exemple dans la page manage-sports.php :**

Lorsqu'on affiche la liste des sports depuis la base de données :

```
while ($row = $statement->fetch(PDO::FETCH_ASSOC)) {
    echo "<tr>";
    // Protection XSS : htmlspecialchars() empêche l'injection de code malveillant
    echo "<td>" . htmlspecialchars($row['nom_sport'], ENT_QUOTES, 'UTF-8') . "</td>";
    echo "<td><button onclick='openModifySportForm({$row['id_sport']})'>Modifier</button></td>";
    echo "<td><button onclick='deleteSportConfirmation({$row['id_sport']})'>Supprimer</button></td>";
    echo "</tr>";
}
```

- Ici, htmlspecialchars() empêche un utilisateur d'injecter du JavaScript dans le nom d'un sport.

**Exemple dans le formulaire de modification (modify-sport.php) :**

```
<input type="text" name="nomSport" id="nomSport"
    value="<?php echo htmlspecialchars($sport['nom_sport'], ENT_QUOTES, 'UTF-8'); ?>" required>
```

- L'utilisateur peut modifier un sport et tenter d'injecter du code HTML ou JS.
- htmlspecialchars() transforme les caractères spéciaux en entités HTML, empêchant l'exécution du code.

**Exemple dans la page de connexion (login.php) :**

```
if (isset($_SESSION['error'])) {
    echo '<p style="color: red;">' . htmlspecialchars($_SESSION['error'], ENT_QUOTES, 'UTF-8') . '</p>';
    unset($_SESSION['error']);
}
```

- Ici, le message d'erreur affiché côté client est **échappé** pour empêcher un utilisateur de manipuler les messages et injecter du JavaScript.

**Exemple lors de la récupération des données POST :**

```
$nom_sport = filter_input(INPUT_POST, 'nom_sport', FILTER_SANITIZE_SPECIAL_CHARS);
```

- filter_input() nettoie le contenu envoyé par l'utilisateur avant de le stocker ou l'afficher.
- Cela empêche l'injection de balises HTML ou de scripts dangereux.

## 8.4. Protection contre les injections SQL

### Définition de la solution :

Les **injections SQL** sont des attaques où un utilisateur malveillant injecte du code SQL arbitraire dans les requêtes exécutées par une application. L'objectif est de contourner les mécanismes d'authentification, de voler, modifier ou supprimer des données, ou d'exécuter des commandes administratives sur la base de données.
Comment nous protégeons l'application
Dans notre application (gestion des sports et accès administrateurs) :

1. **Utilisation de requêtes préparées avec PDO**
    - Toutes les interactions avec la base de données utilisent des requêtes préparées avec des paramètres nommés.
    - Cela sépare le code SQL des données utilisateur, empêchant l'injection de commandes SQL malveillantes.
2. **Validation et typage des paramètres**
    - Les paramètres sont explicitement typés (PDO::PARAM_STR, PDO::PARAM_INT) lors de leur liaison.
    - Les identifiants provenant de l'URL sont validés avec FILTER_VALIDATE_INT avant utilisation.
3. **Gestion sécurisée des entrées utilisateur**
    - Les données utilisateur sont traitées comme non fiables jusqu'à validation.
    - Les valeurs sont liées aux requêtes via bindParam(), empêchant leur interprétation comme code SQL.

## Extraits de code

**Exemple dans la page add-sport.php :**

```
$query = "INSERT INTO SPORT (nom_sport) VALUES (:param_nom_sport)";
$statement = $connexion->prepare($query);
$statement->bindParam(":param_nom_sport", $nom_sport, PDO::PARAM_STR);
$statement->execute();
```

- Ici, le nom du sport est lié comme paramètre séparé, empêchant son interprétation comme code SQL.

**Exemple dans la page delete-sport.php :**

```
$id_sport = filter_input(INPUT_GET, 'id_sport', FILTER_VALIDATE_INT);
// Validation de l'ID
if ($id_sport === false) {
    $_SESSION['error'] = "ID du sport invalide.";
    header("Location: manage-sports.php");
    exit();
}

$sql = "DELETE FROM SPORT WHERE id_sport = :param_id_sport";
$statement = $connexion->prepare($sql);
$statement->bindParam(':param_id_sport', $id_sport, PDO::PARAM_INT);
$statement->execute();
```

- L'ID est validé comme entier avant utilisation.
- Le paramètre est typé comme INT, empêchant l'injection SQL via l'identifiant.

**Exemple dans modify-sport.php :**

```
$queryCheck = "SELECT id_sport FROM SPORT WHERE nom_sport = :nomSport AND
   id_sport <> :param_id_sport";
$statementCheck = $connexion->prepare($queryCheck);
$statementCheck->bindParam(":nomSport", $nomSport, PDO::PARAM_STR);
$statementCheck->bindParam(":param_id_sport", $id_sport, PDO::PARAM_INT);
$statementCheck->execute();
```

- Deux paramètres sont liés séparément avec leurs types appropriés.
- L'opérateur <> est utilisé de manière sécurisée avec des paramètres préparés.

**Exemple dans manage-sports.php :**

```
$query = "SELECT * FROM SPORT ORDER BY nom_sport";
$statement = $connexion->prepare($query);
$statement->execute();
```

- Même pour les requêtes sans paramètre direct, l'utilisation de PDO avec prepare() et execute() assure une couche de protection supplémentaire.